

Q.No: 3

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
2 3
3 1
1 4
3 4
```

EXPECTED OUTPUT

```
1
4 1
```

Test Case 2

INPUT (STDIN)

```
4 5
1 3
2 1
1 2
2 4
1 4
```

EXPECTED OUTPUT

```
2
4 3
3 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m--)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100005

typedef struct Edge {
    int to;
    int next;
} Edge;
```

```
Edge graph[MAX * 2];
int head[MAX];
int edgeCount = 0;

Edge reverseGraph[MAX * 2];
int reverseHead[MAX];
int reverseEdgeCount = 0;

int visited[MAX];
int order[MAX];
int orderCount = 0;
int component[MAX];
int componentCount = 0;

int stack[MAX];
int stackTop;

void addEdge(int u, int v)
{
    graph[edgeCount].to = v;
    graph[edgeCount].next = head[u];
    head[u] = edgeCount++;
}

void addReverseEdge(int u, int v)
{
    reverseGraph[reverseEdgeCount].to = v;
    reverseGraph[reverseEdgeCount].next = reverseHead[u];
    reverseHead[u] = reverseEdgeCount++;
}

void dfs1(int start)
{
    int node, e, nextNode;

    stackTop = 0;
    stack[stackTop++] = start;

    while (stackTop > 0)
    {
        node = stack[stackTop - 1];

        if (!visited[node])
        {
            visited[node] = 1;
        }

        int found = 0;

        for (e = head[node]; e != -1; e = graph[e].next)
        {
            nextNode = graph[e].to;

            if (!visited[nextNode])
```

```
{
stack[stackTop++] = nextNode;
found = 1;
break;
}
}

if (!found)
{
stackTop--;
order[orderCount++] = node;
}
}
}

void dfs2(int start, int comp)
{
int node, e, nextNode;

stackTop = 0;
stack[stackTop++] = start;
component[start] = comp;

while (stackTop > 0)
{
node = stack[--stackTop];

for (e = reverseHead[node]; e != -1; e = reverseGraph[e].next)
{
nextNode = reverseGraph[e].to;

if (component[nextNode] == -1)
{
component[nextNode] = comp;
stack[stackTop++] = nextNode;
}
}
}
}

int main()
{
int n, m;

scanf("%d %d", &n, &m);
for (int i = 1; i <= n; i++)
{
head[i] = -1;
reverseHead[i] = -1;
visited[i] = 0;
component[i] = -1;
}
while (m--)
{
int u, v;

scanf("%d %d", &u, &v);
```

```
addEdge(u, v);
addReverseEdge(v, u);
}

/* First DFS */
for (int i = 1; i <= n; i++)
{
    if (!visited[i])
    {
        dfs1(i);
    }
}

for (int i = orderCount - 1; i >= 0; i--)
{
    int node = order[i];

    if (component[node] == -1)
    {
        dfs2(node, componentCount);
        componentCount++;
    }
}

if (componentCount == 1)
{
    printf("0\n");
    return 0;
}

int indegree[MAX];
int outdegree[MAX];

for (int i = 0; i < componentCount; i++)
{
    indegree[i] = 0;
    outdegree[i] = 0;
}

for (int u = 1; u <= n; u++)
{
    int e;

    for (e = head[u]; e != -1; e = graph[e].next)
    {
        int v = graph[e].to;

        if (component[u] != component[v])
        {
            outdegree[component[u]] = 1;
            indegree[component[v]] = 1;
        }
    }
}
```

```
int sources[MAX];
int sinks[MAX];

int sourceCount = 0;
int sinkCount = 0;

for (int i = 0; i < componentCount; i++)
{
    if (indegree[i] == 0)
    {
        sources[sourceCount++] = i;
    }

    if (outdegree[i] == 0)
    {
        sinks[sinkCount++] = i;
    }
}

printf("Ouptut:\n");

int answer = sourceCount > sinkCount
? sourceCount
: sinkCount;

printf("%d\n", answer);
for (int i = 0; i < answer; i++)
{
    int from = sinks[i % sinkCount];
    int to = sources[(i + 1) % sourceCount];

    int fromNode = -1;
    int toNode = -1;
    for (int j = 1; j <= n; j++)
    {
        if (component[j] == from)
        {
            fromNode = j;
            break;
        }
    }

    for (int j = 1; j <= n; j++)
    {
        if (component[j] == to)
        {
            toNode = j;
            break;
        }
    }

    printf("%d %d\n", fromNode, toNode);
}
```

```
return 0;  
}
```

Output:

Test 1:

```
● aadityajaiswal@Aad  
4 5  
1 2  
2 3  
3 1  
1 4  
3 4  
Output:  
1  
4 1  
● aadityajaiswal@Aad
```

Test2:

```
● aadityajaiswal@Aad  
4 5  
1 3  
2 1  
1 2  
2 4  
1 4  
Output:  
2  
3 1  
4 1  
● aadityajaiswal@Aad
```